

Event Booking Web Application

Django | SQLite/PostgreSQL | Django Templates

1. Introduction

The Event Booking System is a full-stack web application developed using the Django framework with Django templates for the presentation layer. The system enables registered users to browse published events, search by category or date, view seat availability, and book tickets online. Staff users manage events and categories through a dedicated management interface. The project demonstrates CRUD operations, authentication, form validation, email notifications, and deployment readiness.

2. Problem Statement

Manual event ticketing is slow, error-prone, and difficult to scale. Organizers struggle to track capacity, while attendees lack a single place to discover events and receive booking confirmations. There is a need for an accessible, web-based platform that centralizes event listings, enforces capacity limits, and automates confirmations.

3. Objectives

- Allow users to register, log in, and manage their profile via Django auth.
- Display events with date, time, location, category, and capacity.
- Indicate when events are sold out based on confirmed bookings.
- Enable search/filter by keyword, category, and date.
- Process ticket bookings with validation and confirmation emails.
- Provide staff CRUD for events and categories.
- Deliver a responsive, attractive user interface.
- Deploy to Render with automated migrations and static files.

4. Methodology

An iterative software development approach was used: requirements analysis, database design, implementation of Django models/forms/views/URLs, template and CSS design, unit testing with Django TestCase, and deployment configuration. Version control is maintained on GitHub. SQLite is used locally; PostgreSQL on Render via DATABASE_URL for production persistence.

5. System Requirements

Functional: user registration/login; browse and search events; view event details; book tickets (authenticated); email confirmation; view/cancel bookings; staff event and category CRUD.

Non-functional: responsive UI; form validation; secure password rules; HTTPS on production; static file serving via WhiteNoise.

Technical: Python 3.12+, Django 5.x, Gunicorn, SQLite (dev), PostgreSQL (Render).

6. System Actors

Guest: browses events, must register to book.

Registered User: books tickets, views/cancels own bookings.

Staff/Organizer: creates, updates, deletes events and categories.

Administrator: Django admin for full data management.

7. System Design - Database Models

User (Django auth): username, email, password.

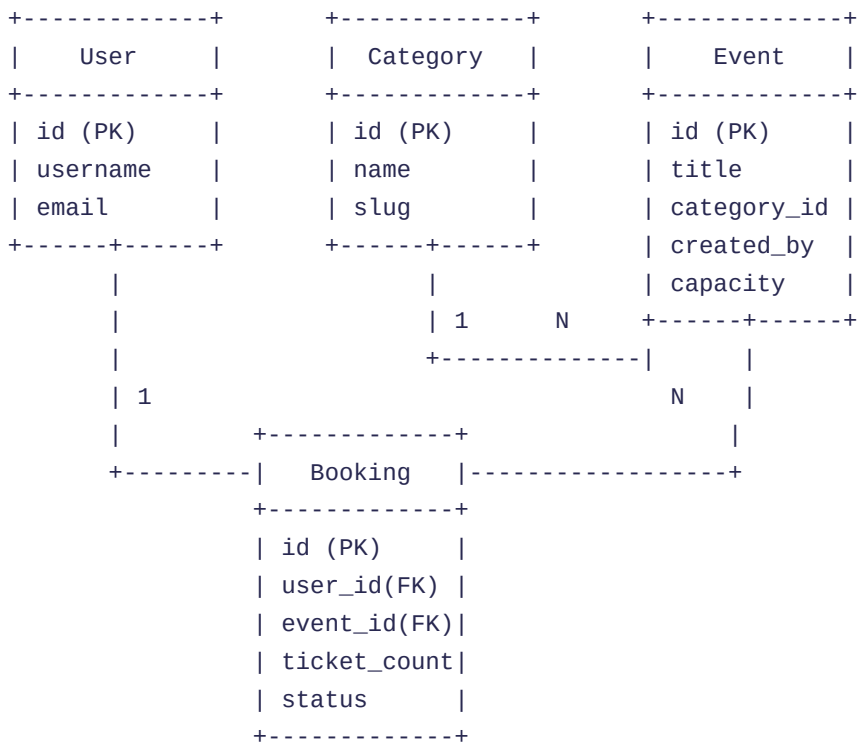
Category: name, slug, description.

Event: title, slug, description, category (FK), event_date, event_time, location, capacity, image, is_published, created_by (FK User).

Booking: user (FK), event (FK), ticket_count, confirmation_code, status, booked_at.

Derived logic: $\text{tickets_sold} = \text{sum of confirmed booking ticket_count}$; $\text{seats_remaining} = \text{capacity} - \text{tickets_sold}$; is_full when $\text{seats_remaining} \leq 0$.

8. Entity Relationship Diagram



9. Screenshots

Screenshots should be captured from the running application showing: home page with event grid and search; event detail with availability; registration/login; booking confirmation; my bookings list; staff manage dashboard; and event create form. After deployment, include the live Render URL in the report appendix.

10. Testing Results

Automated tests (python manage.py test) cover:

- Home page and event listing
- Search by category and date
- User registration and login
- Booking requires authentication
- Successful booking and confirmation email
- Sold-out detection when capacity reached
- Duplicate booking prevention
- Booking cancellation restores seats
- Staff event creation and manage access control
- Unique confirmation code generation

All tests are expected to pass before deployment.

11. Challenges Faced

- Enforcing capacity across concurrent bookings using aggregated ticket counts.
- Conditional unique constraint for one active booking per user per event.
- Render ephemeral filesystem: PostgreSQL used in production while SQLite locally.
- Email delivery in production requires SMTP environment variables.
- Static file serving configured with WhiteNoise and collectstatic.

12. Conclusion

The Event Booking System successfully meets the project requirements: Django MVC architecture, SQLite/PostgreSQL storage, template-based frontend, CRUD for staff, user authentication, search, capacity tracking, and email confirmations. The interface uses modern typography and responsive layout to attract users.

13. Recommendations

- Add payment gateway integration for paid events.
- Implement QR codes on confirmation emails.
- Add waitlist when events are full.
- Use Celery for async email and reminders.
- Add API layer (Django REST Framework) for mobile apps.
- Enable social authentication (Google/GitHub OAuth).